

Command-Line Agents

Moving past the chat window

What a Coding Agent Is

- A program that reads your files, runs commands, and edits in place
- It works in a loop, not a single reply
- You stay in the chair -- it proposes, you approve

```
+-----+
|  read files / repo  |
+-----+-----+
|
|  v
+-----+-----+
|  run a command      |
+-----+-----+
|
|  v
+-----+-----+
|  edit a file        |
+-----+-----+
|
|  v
+-----+-----+
|  check, repeat      |
+-----+-----+
```

Chat vs Terminal

- Chat sees only what you paste -- the agent sees the whole repo
- Chat hands back text -- the agent hands back a diff you can review
- Same prompt plus same rules gives the same structure every time

Chat window

paste a file

->

copy reply back

->

no memory

->

no commands

->

Coding agent

reads **any** file

edits **in** place

project rules

runs the **build**

CLAUDE.md

- A markdown file the agent reads on every run
- Holds project rules: style, conventions, what not to touch
- Encode your taste once, reuse it on every task

Lectures

- * Use a two-cols layout
- * Titles short, no "Best Practices"
- * Max three bullets per slide
- * Italics for emphasis, not bold
- * Asterisk bullets, no numbered lists

Permissions

- The agent asks before it edits or runs commands
- You allowlist the safe, repetitive calls
- Nothing lands without your say-so

```
Edit  meta/lectures/37_ai.md  [y/n]
```

```
Bash  just gen-pdfs          [y/n]
```

```
Bash  rm -rf meta/          [n]
```

Skills

- Canned workflows you trigger with a slash
- Bundle a multi-step task behind one command
- The course uses `/review-lecture` to grade slides against a transcript

```
/review-lecture
```

```
/code-review
```

```
/gen-quiz
```

```
/verify
```

The Stance

- Code is cheap to generate -- understanding it is the expensive part
- Work in small steps, not giant changelists no one can read
- Stay the sorcerer, not the apprentice

```
generate -> read -> accept
```

```
generate -> read -> reject
```

```
generate -> read -> revise
```

```
never:
```

```
generate -> commit -> hope
```

Review Every Diff

- The agent is fast and confident, including when it is wrong
- A diff is the unit of trust -- read it before you accept
- `git` is the safety net: nothing is permanent until you commit

C's Design Principles

- * Trust the programmer
- + * `_Trust the programmer_` -- low-level access
 - * One-to-one with assembly where possible
- + * No runtime safety net

DEMO: Cold Open

- Open the course repo and ask the agent what the project is
- It reads CLAUDE.md and the tree, then answers
- Shows the loop, the file access, and the project rules in one shot

Summary

- A coding agent reads files, runs commands, and edits in a loop
- The terminal beats chat: full repo context and reviewable diffs
- CLAUDE.md encodes your rules so output matches your style
- Permissions keep you in control of edits and commands
- Skills wrap multi-step workflows behind a slash command
- Code is cheap; understanding is the bottleneck
- Work incrementally and read every diff before accepting
- git is the undo button -- nothing lands until you commit

Command-Line Agents

Moving past the chat window
